

Clinic AI Assistant — Day 3 Post-Mortem: AI Integration

Objective: Connect the backend chat system to a real Large Language Model (LLM) and establish a clean AI service layer while preserving a scalable multi-tenant architecture.

System Flow:

Browser Chat UI → POST /chat?tenant= → routes/chat.py → services/ai_agent.py → OpenAI API → LLM Response → Frontend UI

Key Architectural Rule:
Only one component communicates with the LLM: services/ai_agent.py

File	Role	Notes
app/services/ai_agent.py	AI communication layer	Handles OpenAI requests and prompt creation
app/routes/chat.py	Chat endpoint	Receives user messages and delegates to AI service
.env	Environment configuration	Stores OPENAI_API_KEY securely
app/main.py	Application bootstrap	Loads environment variables and mounts routes

Key Implementation Decisions:

- AI Service Isolation
All model communication centralized inside services/ai_agent.py.
- Config Driven Tenants
Business behavior controlled through configuration instead of code.
- Environment Variable Security
API keys stored only in backend .env and never exposed to frontend.
- Error Handling Layer
API failures return safe fallback responses instead of crashing the application.

System Status After Day 3:

- ✓ Frontend chat interface operational
- ✓ Backend chat endpoint functional
- ✓ Multi-tenant configuration working
- ✓ AI service layer implemented
- ✓ OpenAI API connected
- ✓ Error handling verified

Next Development Stage (Day 4):

- Introduce clinic service catalogue configuration
- Map patient symptoms to available services
- Display indicative pricing
- Begin appointment booking workflow